

Cześć III

Grupa 8 (TP):

Wprowadzenie: 20 maja 2026

Termin wysłania Części III: 3 czerwca 2026

Termin oddania: 3 czerwca 2026

Grupy 3,6 (TN):

Wprowadzenie: 27 maja 2026

Termin wysłania Części III: 10 czerwca 2026

Termin oddania: 10 czerwca 2026

Cel projektu

Na drugiej liście poznaliście zjawisko przeuczania (Overfitting) i "Czarnego Łabędzia". W tej finałowej części semestru nauczycie się, jak przed tymi zjawiskami bronić. Zaczniemy od nałożenia matematycznego "kagańca" na modele (Regularyzacja). Następnie przejdziemy do potężnej koncepcji modeli zespołowych (Ensemble Learning), budując rozwiązania krok po kroku: od mechanizmu Baggingu, przez kaskadowy Stacking i sekwencyjny Boosting, aż po zaprogramowanie własnej, uproszczonej architektury Mixture of Experts (MoE), która jest dziś fundamentem działania największych modeli AI (np. GPT-4).

Część implementacyjna

Zakres prac na ocenę 3.0 (Regularyzacja):

- **Regularyzacja L1/L2:** Zaimplementuj modele liniowe z wbudowaną karą: Lasso (L1) oraz Ridge (L2). Przetestuj różne siły regularyzacji (parametr α). Pokaż na wykresie lub tabeli, że Lasso faktycznie potrafi wyzerować wagi (działa jako selektor cech), a Ridge jedynie ściąga je w stronę zera. Implementację wykonaj samodzielnie, rozszerzając metodę treningu Gradient Descent.
- **Regularyzacja drzew decyzyjnych:** Na Liście 2 wykorzystano parametr *max_depth*. Zbuduj model drzewa i zbadaj wpływ innych parametrów strukturalnych:
 - *min_samples_split* i *min_samples_leaf* (wymagania co do minimalnej liczby próbek w węzłach),
 - *max_features* (losowanie podzbioru cech podczas podziału).
- **Znajdź "sweet spot" dla "zoverfittowanego" modelu.** Udowodnij, że zregularyzowane drzewo i zregularyzowana regresja lepiej radzą sobie na zbiorze testowym niż ich wersje bazowe. Podobnie jak na Liście nr 2, pokaż wykresy dla wyników na zbiorach train/test, ale tym razem modyfikując parametry regularyzacji.

Zakres prac na ocenę 3.5 (Bagging)

Bagging (Bootstrap Aggregating) to łączenie identycznych modeli pracujących równolegle, aby zredukować wariancję.

- *scikit-learn*: Użyj gotowego modelu `RandomForestClassifier`, porównaj jego wynik z pojedynczym drzewem.
- *Własna implementacja*: Napisz od zera własną funkcję/klasę realizującą Bagging na zwykłych drzewach decyzyjnych. Twój kod musi:
 - Sampling (Bootstrap): Wylosować N podzbiorów treningowych ze zwracaniem (niektóre wiersze się powtarzają, inne są pomijane).
 - Trening: Wytrenować N oddzielnych "słabych" drzew, każde na swoim wylosowanym podziorze.
 - Agregacja (Voting): W fazie predykcji zebrać głosy od wszystkich N drzew dla testowej próbki i zastosować twarde lub miękkie głosowanie.
- Zaprezentuj wyniki działania modeli dla każdego wariantu.

Zakres prac na ocenę 4.0 (Stacking)

Bagging łączy modele tego samego typu. Stacking pozwala połączyć heterogeniczne (zupełnie różne) algorytmy, tworząc z nich "komitet".

- Poziom 0 (modele bazowe): Wybierz co najmniej 3 różnie działające modele (np. Drzewo Decyzyjne, zregularyzowana Regresja (Lasso/Ridge) z etapu 3.0 oraz K-Najbliższych Sąsiadów).
- Poziom 1 (meta-model): Zamiast uśredniać głosy "w ciemno", wytrenuj kolejny model (np. prostą regresję Logistyczną/Liniową), który nauczy się, któremu z modeli Level-0 należy ufać. Cechami wejściowymi (X) dla meta-modelu mają być predykcje wygenerowane przez modele z Poziomu 0.

Ważne: Aby uniknąć wycieku danych, do wygenerowania cech dla meta-modelu na zbiorze treningowym użyj techniki `cross_val_predict` ze *scikit-learn* (lub ręcznie wydziel osobny zbiór walidacyjny dla Poziomu 1).

Zakres prac na ocenę 4.5 (Boosting)

Podczas gdy Bagging redukuje wariancję, Boosting redukuje błąd obciążenia (Bias), ucząc modele sekwencyjnie.

- Zaimplementuj prosty algorytm z rodziny boosting. *HINT. Skup się na problemie regresji, który w tym wypadku jest łatwiejszy od klasyfikacji). Niemniej jednak, klasyfikacja nadal jest możliwa.*
- Skonfiguruj go tak, aby używał np. 100 bardzo płytkich drzew (np. `max_depth=3`).
- Porównaj wyniki algorytmu pozostałymi modelami ze wcześniejszych etapów.

Pseudokod dla uproszczonej wersji algorytmu boostingowego:

Pseudo-algorytm Gradient Boosting:

1. **Inicjalizacja:** Zbuduj najprostszy możliwy model bazowy $F_0(X)$ (np. przewidyjący po prostu średnią wartość z całego zbioru Y).
2. **Dla $m = 1$ do M (gdzie M to liczba drzew, np. 100):**
 - **Krok A:** Oblicz błędy (tzw. wartości resztkowe / *residuals*) obecnego modelu dla każdej próbki i :
$$reszta_i = Y_i - F_{m-1}(X_i)$$
 - **Krok B:** Wytrenuj nowe, "słabe" i płytkie drzewo decyzyjne $h_m(X)$ (np. `max_depth=3`).
UWAGA: Jako zmiennej docelowej (targetu) dla tego drzewa nie używaj oryginalnego Y , lecz obliczonych przed chwilą reszt! Model uczy się przewidywać, o ile pomylił się poprzedni model.
 - **Krok C:** Zaktualizuj główny model, dodając do niego nowe drzewo, opcjonalnie osłabione przez parametr `learning_rate` (aby nauka postępowała mniejszymi, stabilniejszymi krokami):
$$F_m(X) = F_{m-1}(X) + (learning_rate \times h_m(X))$$
3. **Zakończenie:** Ostateczna predykcja dla nowych danych to wynik z modelu $F_M(X)$.

Klasyczny algorytm Gradient Boosting (dla chętnych, nie jest wymagany do implementacji w ramach listy):

Klasyczny Algorytm Gradient Boosting Machine (GBM):

Niech $L(y, F(x))$ oznacza wybraną różniczkowalną funkcję straty (np. Błąd Średniokwadratowy - MSE).

1. **Inicjalizacja:** Zainicjuj model stałą wartością, która globalnie minimalizuje funkcję straty dla całego zbioru (dla MSE będzie to średnia z Y , dla MAE będzie to mediana):
$$F_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$$
2. **Dla $m = 1$ do M (liczba drzew):**
 - **Krok A (Pseudo-reszty):** Oblicz tzw. pseudo-reszty, czyli ujemny gradient funkcji straty dla każdej próbki i . Mówi to nam, w jakim kierunku (i jak mocno) model musi skorygować predykcję:
$$r_{im} = - \left[\frac{\partial L(y_i, F(x))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$
(Zauważ: Jeśli L to MSE, powyższa pochodna upraszcza się idealnie do $y_i - F_{m-1}(x_i)$).
 - **Krok B (Struktura drzewa):** Wytrenuj nowe, płytkie drzewo decyzyjne na zbiorze docelowym r_{im} . Drzewo to podzieli przestrzeń na regiony decyzyjne (liście) oznaczone jako R_{jm} (dla $j = 1 \dots J$, gdzie J to liczba liści).
 - **Krok C (Optymalizacja Wag Liści):** Dla każdego utworzonego liścia j , oblicz optymalną wagę wyjściową γ_{jm} , która zminimalizuje oryginalną funkcję straty po dodaniu jej do obecnego modelu:
$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$$
 - **Krok D (Aktualizacja Modelu):** Zaktualizuj główny model dodając nowe drzewo. Każdy wiersz wpada do odpowiedniego liścia i otrzymuje wyliczoną wagę γ , dodatkowo pomnożoną przez współczynnik uczenia (`learning_rate`, ν):
$$F_m(x) = F_{m-1}(x) + \nu \sum_{j=1}^J \gamma_{jm} I(x \in R_{jm})$$
3. **Zakończenie:** Gotowy zespół drzew to $F_M(x)$.

Zakres prac na ocenę 5.0 ("Mixture of Experts")

W Stackingu meta-model nakłada globalne wagi na algorytmy. W architekturze MoE idziemy o krok dalej: specjalna sieć-bramka (Gating Network) decyduje dynamicznie, do którego eksperta wysłać dany przypadek.

Zbuduj ten system od zera:

- Stworzenie bramki (routera): Użyj algorytmu klasteryzacji (np. KMeans dla $K=3$ klastrów), aby podzielić zbiór treningowy na 3 naturalne grupy. Następnie wytrenuj klasyfikator (np. las losowy/boosting), który nauczy się odwzorowywać cechy rekordu na numer klastra. To Twoja bramka.
- Trening ekspertów: Zbierz dane przypisane do klastra 1 i wytrenuj na nich eksperta 1 (np. regresję). To samo zrób dla klastra 2 (ekspert 2) i 3 (ekspert 3). Każdy ekspert staje się wąskim specjalistą dla swojego wycinka danych.
- Wnioskowanie: Dla nowej, testowej próbki:
 - Zapytaj Bramkę o decyzję (odpowiedź powinna być w formie: "Ten wiersz ma cechy Klastra 2").
 - Skieruj cały rekord do predykcji wyłącznie przez Eksperta 2.
 - Sprawdź, czy podzielenie problemu na mniejszych ekspertów zadziałało lepiej niż model próbujący uogólnić cały zbiór jednocześnie.

Część raportowa

Raport powinien zawierać sensowne metryki w formie tabeli wyników dla wszystkich zbudowanych algorytmów, która umożliwi ich porównanie. Pamiętaj, żeby dane wyniki były przeliczone na zbiorze testowym, który zawiera dane niewidoczne podczas treningu.

Dodatkowo:

- Regularyzacja: Jaki efekt na współczynniki miał parametr Lasso, a jaki Ridge? Jak zmienił się błąd na zbiorze testowym w porównaniu z modelami surowymi?
- Bagging: Krótko opisz, dlaczego losowanie ze zwracaniem jest absolutnie kluczowe w Twojej własnej implementacji Baggingu. Co stałoby się, gdybyśmy wylosowali 100 drzew na dokładnie tych samych podzbiorach?
- Stacking i Boosting: Co poradziło sobie lepiej na Twoich danych? Czy meta-model w Stackingu "odkrył", że któryś z modeli Poziom 0 się myli i przestał brać jego predykcje pod uwagę?
- Mixture of Experts: Skomentuj logikę podziału przestrzeni w Twoim systemie MoE. Czy klasteryzacja KMeans podzieliła Twoich pacjentów/diamenty/klientów w sensowny sposób? Jakie korzyści architektoniczne płyną z delegowania wnioskowania tylko do 1 konkretnego eksperta w porównaniu do odpalania pełnego 100-drzewnego lasu losowego.

Dodatkowo

Poniższe linki i artykuły pomagają w zrozumieniu zagadnień oraz je rozszerzają:

Bagging, Boosting, Stacking (GeeksForGeeks)

<https://www.geeksforgeeks.org/machine-learning/bagging-vs-boosting-vs-stacking/>

Czym jest Wzmocnienie Gradientowe (Gradient Boosting)? (Miroslaw Mamczur)

<https://miroslawmamczur.pl/czym-jest-wzmocnienie-gradientowe-gradient-boosting-i-dlaczego-jest-taki-dobry/>

All You Need to Know about Gradient Boosting (Medium)

<https://medium.com/data-science/all-you-need-to-know-about-gradient-boosting-algorithm-part-1-regression-2520a34a502>

Mixture of Experts Explained (Hugging Face)

<https://huggingface.co/blog/moe>